

Sequential Decision

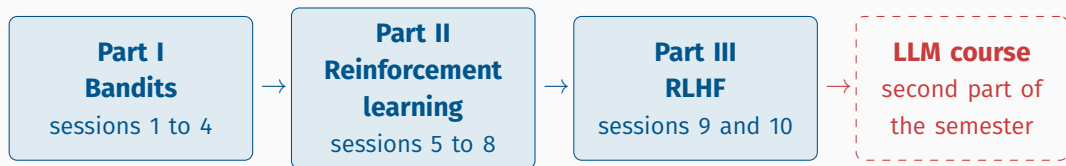
Session 1: The K -armed bandit

Igor Colin

October 2026

ESIEE Paris

The course



A 3-hour session

- 1h lecture
- 40 min tutorial: computations by hand
- 1h20 lab: implementation in Python

Grading

- Lab checkpoints (sessions 4 and 8): 25 %
- Project in pairs, with oral defense: 35 %
- Written exam: 40 %

Sequential learning

Supervised learning

- A dataset fixed in advance
- Learn first, then predict
- Errors are measured after training

Agent

Sequential learning

- Data arrive as decisions are made
- **Decisions choose the data:** we only see the outcome of the action we took
- Mistakes made while learning count

Environment

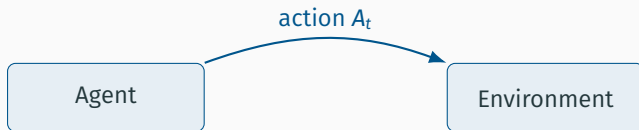
Sequential learning

Supervised learning

- A dataset fixed in advance
- Learn first, then predict
- Errors are measured after training

Sequential learning

- Data arrive as decisions are made
- **Decisions choose the data**: we only see the outcome of the action we took
- Mistakes made while learning count



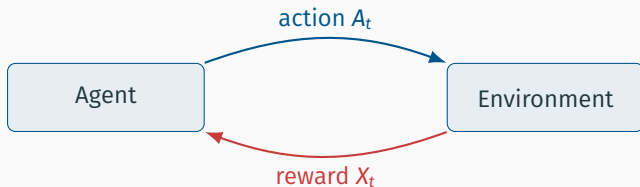
Sequential learning

Supervised learning

- A dataset fixed in advance
- Learn first, then predict
- Errors are measured after training

Sequential learning

- Data arrive as decisions are made
- **Decisions choose the data**: we only see the outcome of the action we took
- Mistakes made while learning count



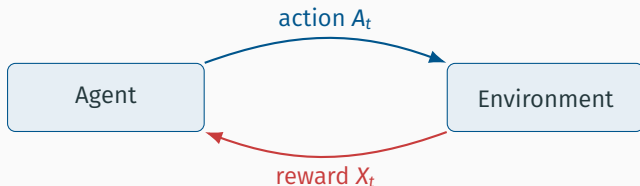
Sequential learning

Supervised learning

- A dataset fixed in advance
- Learn first, then predict
- Errors are measured after training

Sequential learning

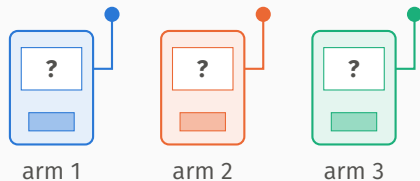
- Data arrive as decisions are made
- **Decisions choose the data**: we only see the outcome of the action we took
- Mistakes made while learning count



repeat for $t = 1, \dots, T$

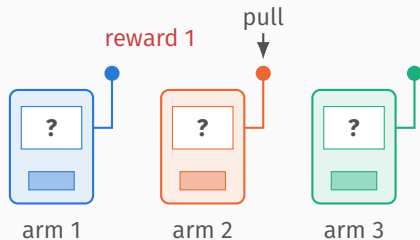
The multi-armed bandit

The multi-armed bandit



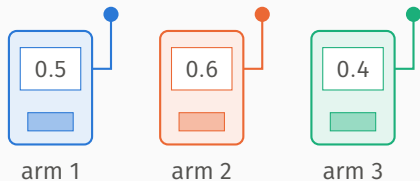
- A slot machine is a “one-armed bandit”

The multi-armed bandit



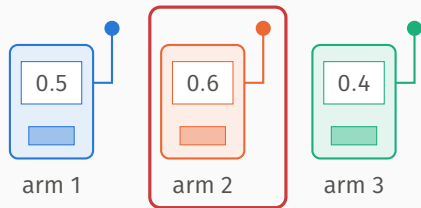
- A slot machine is a “one-armed bandit”
- Pulling an arm gives a **random reward**

The multi-armed bandit



- A slot machine is a “one-armed bandit”
- Pulling an arm gives a **random reward**
- Arm a pays μ_a on average, but the μ_a are **unknown**

The multi-armed bandit



- A slot machine is a “one-armed bandit”
- Pulling an arm gives a **random reward**
- Arm a pays μ_a on average, but the μ_a are **unknown**
- Goal: earn as much as possible, which requires finding the best arm *while* playing

Applications

Application	Arms	Reward
Clinical trials	treatments	the patient recovers
Online advertising	ads	the visitor clicks
A/B testing	versions of a web page	the visitor buys
Recommendation	articles, songs, videos	the user reads, listens, watches
Dynamic pricing	prices	revenue
Hyperparameter tuning	configurations	validation score
LLM applications	models, prompts	user feedback (thumbs up)

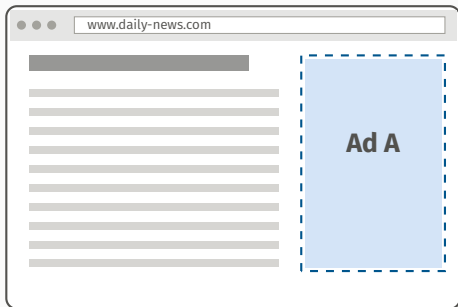
In all cases, the best option is unknown and every trial of a bad option has a cost.

Example: online advertising



- **Round t :** visitor t loads the page
- **Arms:** $K = 3$ ads, A_t is the ad shown to visitor t
- **Reward:** $X_t = 1$ if the visitor clicks, 0 otherwise
- **Unknown means:** μ_a is the click-through rate (CTR) of ad a
- **Horizon:** $T = 1,000,000$ visitors this month

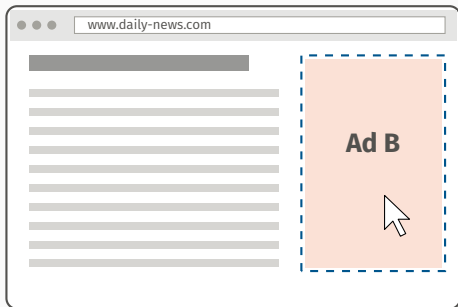
Example: online advertising



no click: $X_t = 0$

- **Round t :** visitor t loads the page
- **Arms:** $K = 3$ ads, A_t is the ad shown to visitor t
- **Reward:** $X_t = 1$ if the visitor clicks, 0 otherwise
- **Unknown means:** μ_a is the click-through rate (CTR) of ad a
- **Horizon:** $T = 1,000,000$ visitors this month

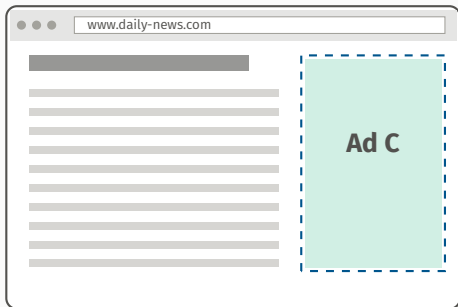
Example: online advertising



click: $X_t = 1$

- **Round t :** visitor t loads the page
- **Arms:** $K = 3$ ads, A_t is the ad shown to visitor t
- **Reward:** $X_t = 1$ if the visitor clicks, 0 otherwise
- **Unknown means:** μ_a is the click-through rate (CTR) of ad a
- **Horizon:** $T = 1,000,000$ visitors this month

Example: online advertising



no click: $X_t = 0$

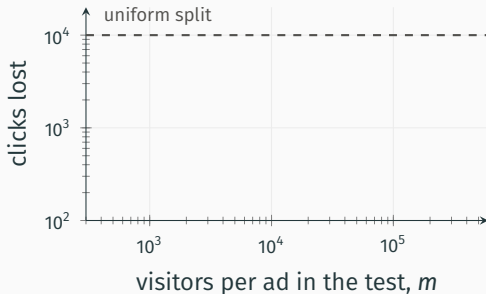
- **Round t :** visitor t loads the page
- **Arms:** $K = 3$ ads, A_t is the ad shown to visitor t
- **Reward:** $X_t = 1$ if the visitor clicks, 0 otherwise
- **Unknown means:** μ_a is the click-through rate (CTR) of ad a
- **Horizon:** $T = 1,000,000$ visitors this month

Example: length of an A/B/C test

Suppose the true CTRs are A: 4 %, B: 5 %, C: 3 %. Always showing B gives 50,000 clicks: we count the **clicks lost** with respect to this.

A test shows each ad to m visitors, then the winner to everyone else.

Strategy	Clicks lost
Uniform split, all month	10,000

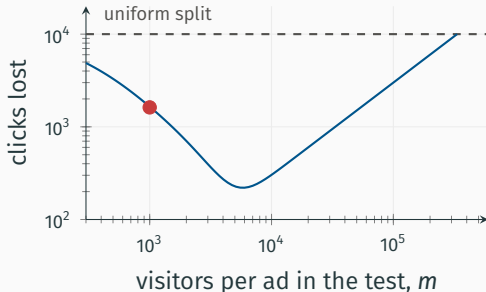


Example: length of an A/B/C test

Suppose the true CTRs are A: 4 %, B: 5 %, C: 3 %. Always showing B gives 50,000 clicks: we count the **clicks lost** with respect to this.

A test shows each ad to m visitors, then the winner to everyone else.

Strategy	Clicks lost
Uniform split, all month	10,000
Test with $m = 1,000$	$\approx 1,600$



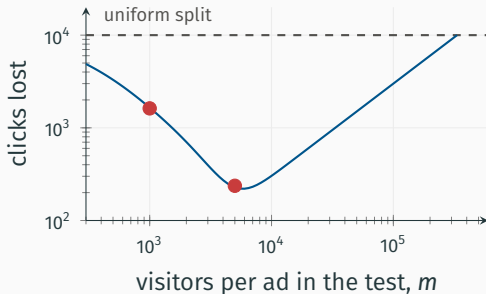
- Too short: the winner is often wrong (A beats B in 14 % of the tests)

Example: length of an A/B/C test

Suppose the true CTRs are A: 4 %, B: 5 %, C: 3 %. Always showing B gives 50,000 clicks: we count the **clicks lost** with respect to this.

A test shows each ad to m visitors, then the winner to everyone else.

Strategy	Clicks lost
Uniform split, all month	10,000
Test with $m = 1,000$	$\approx 1,600$
Test with $m = 5,000$	≈ 240



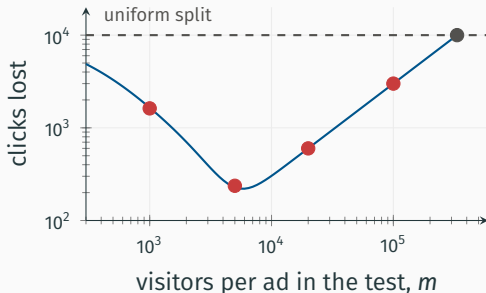
- Too short: the winner is often wrong (A beats B in 14 % of the tests)

Example: length of an A/B/C test

Suppose the true CTRs are A: 4 %, B: 5 %, C: 3 %. Always showing B gives 50,000 clicks: we count the **clicks lost** with respect to this.

A test shows each ad to m visitors, then the winner to everyone else.

Strategy	Clicks lost
Uniform split, all month	10,000
Test with $m = 1,000$	$\approx 1,600$
Test with $m = 5,000$	≈ 240
Test with $m = 20,000$	≈ 600
Test with $m = 100,000$	3,000



- Too short: the winner is often wrong (A beats B in 14 % of the tests)
- Too long: the test costs $0.01 + 0.02 = 0.03$ click per visitor per ad, so $0.03 m$ in total; $m = 333,333$ is the uniform split

Example: clinical trials

Fixed randomization (50/50)



 treatment A  treatment B (better)

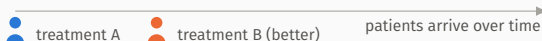
- **Round:** a patient enters the trial
- **Arms:** treatments; **reward:** recovery
- Every patient given the worse treatment is a unit of regret

Example: clinical trials

Fixed randomization (50/50)



Adaptive allocation



- **Round:** a patient enters the trial
- **Arms:** treatments; **reward:** recovery
- Every patient given the worse treatment is a unit of regret
- Thompson (1933) introduced the first bandit algorithm for exactly this question; response-adaptive designs are used in modern trials
- Tension: statistical evidence for future patients vs. welfare of current ones

Stochastic K -armed bandit

Distributions $\nu_1, \dots, \nu_K$ **unknown** to the agent, with means $\mu_1, \dots, \mu_K$. For $t = 1, \dots, T$:

1. the agent chooses an arm $A_t \in \{1, \dots, K\}$ based on the history $A_1, X_1, \dots, A_{t-1}, X_{t-1}$;
2. it observes the reward X_t , drawn from ν_{A_t} , and **only this one**.

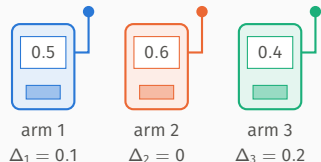
- A **policy** is the rule mapping histories to choices (it may be randomized)
- T is the **horizon**
- **Bandit feedback**: the rewards of the arms we did not play remain unknown
- Typical case: Bernoulli arms, $X_t \in \{0, 1\}$ with $\mathbb{P}(X_t = 1) = \mu_{A_t}$

- Best mean: $\mu^* = \max_a \mu_a$
- Gap of arm a : $\Delta_a = \mu^* - \mu_a \geq 0$
- Number of pulls of arm a :

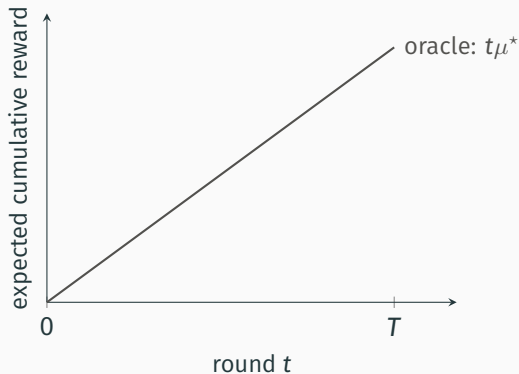
$$N_a(t) = \sum_{s=1}^t \mathbf{1}\{A_s = a\}$$

- Empirical mean of arm a :

$$\hat{\mu}_a(t) = \frac{1}{N_a(t)} \sum_{s=1}^t X_s \mathbf{1}\{A_s = a\}$$



$$K = 3, \mu^* = 0.6$$

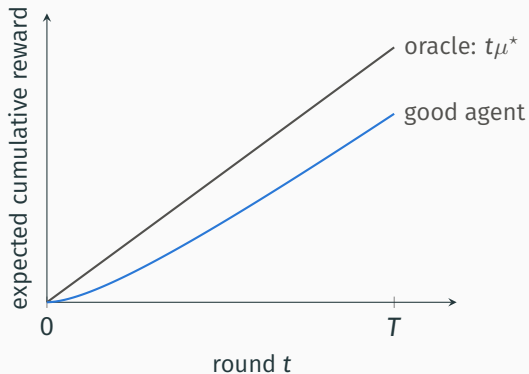


The **oracle** knows the best arm and plays it every round.

Regret

$$R_T = T\mu^* - \mathbb{E}\left[\sum_{t=1}^T X_t\right]$$

- Maximizing reward $\Leftrightarrow$ minimizing regret



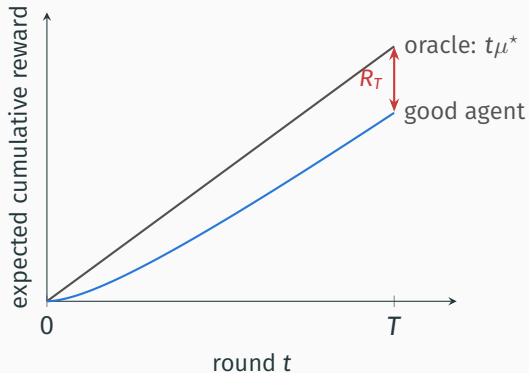
The **oracle** knows the best arm and plays it every round.

Regret

$$R_T = T\mu^* - \mathbb{E}\left[\sum_{t=1}^T X_t\right]$$

- Maximizing reward $\Leftrightarrow$ minimizing regret

Regret



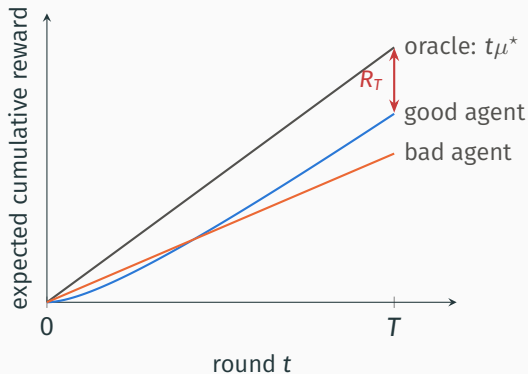
The **oracle** knows the best arm and plays it every round.

Regret

$$R_T = T\mu^* - \mathbb{E}\left[\sum_{t=1}^T X_t\right]$$

- Maximizing reward $\Leftrightarrow$ minimizing regret

Regret



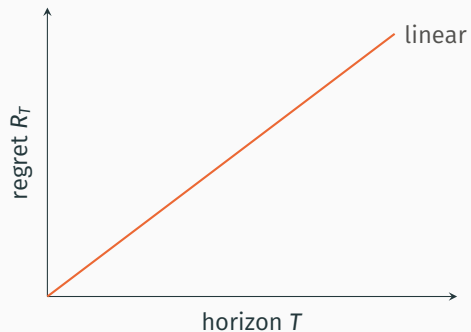
The **oracle** knows the best arm and plays it every round.

Regret

$$R_T = T\mu^* - \mathbb{E}\left[\sum_{t=1}^T X_t\right]$$

- Maximizing reward $\Leftrightarrow$ minimizing regret
- A bad agent loses a constant fraction of each round: linear regret

Sublinear regret



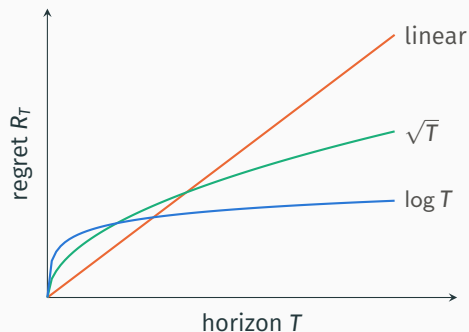
- $R_T \leq T$ for rewards in $[0, 1]$:
linear regret is trivial

Sublinear regret



- $R_T \leq T$ for rewards in $[0, 1]$: linear regret is trivial
- **Goal:** sublinear regret, $R_T/T \rightarrow 0$: the agent eventually plays (almost) only the best arm

Sublinear regret

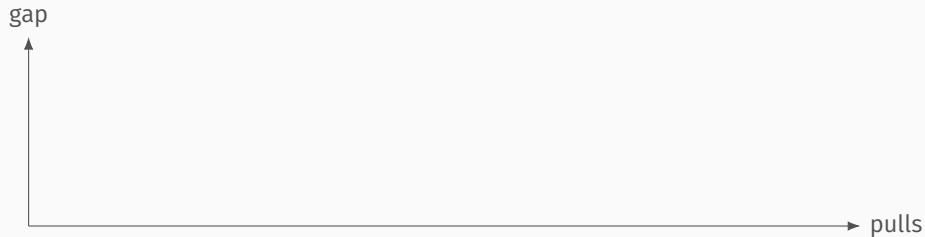


- $R_T \leq T$ for rewards in $[0, 1]$: linear regret is trivial
- **Goal:** sublinear regret, $R_T/T \rightarrow 0$: the agent eventually plays (almost) only the best arm
- $\log T$ is the best possible rate on a fixed instance (session 2)

Regret decomposition

Proposition (proof in the tutorial, exercise 1)

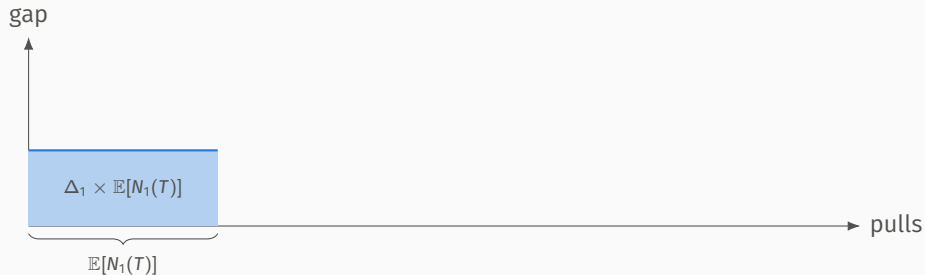
$$R_T = \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)]$$



Regret decomposition

Proposition (proof in the tutorial, exercise 1)

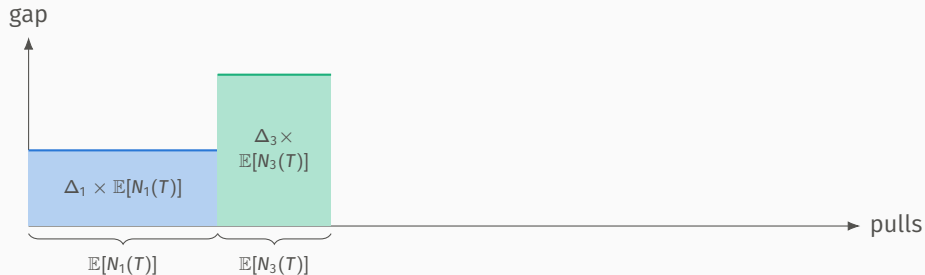
$$R_T = \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)]$$



Regret decomposition

Proposition (proof in the tutorial, exercise 1)

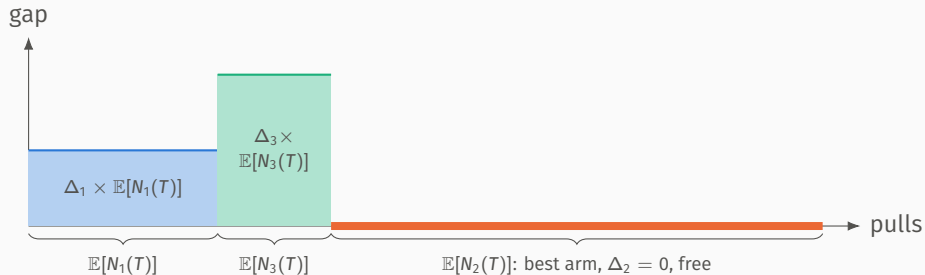
$$R_T = \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)]$$



Regret decomposition

Proposition (proof in the tutorial, exercise 1)

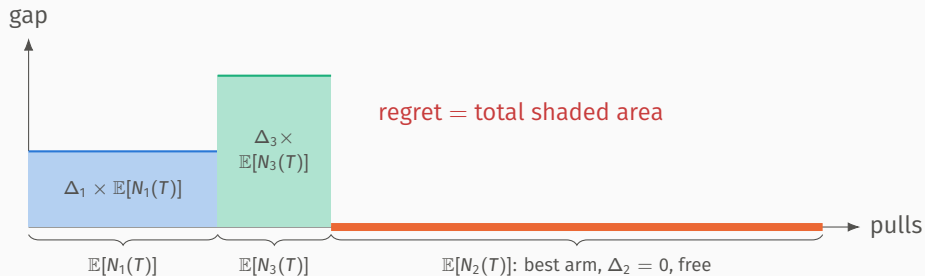
$$R_T = \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)]$$



Regret decomposition

Proposition (proof in the tutorial, exercise 1)

$$R_T = \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)]$$



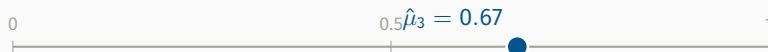
- A good algorithm **rarely pulls suboptimal arms**, without giving up on them too early
- In simulations we plot the **pseudo-regret** $\bar{R}_t = \sum_{s \leq t} \Delta_{A_s}$, whose expectation is R_t

Estimating a mean

Incremental update

No need to store past rewards: if $\hat{\mu}_n$ is the mean of the first n rewards and x_{n+1} is the next one,

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1} (x_{n+1} - \hat{\mu}_n) .$$

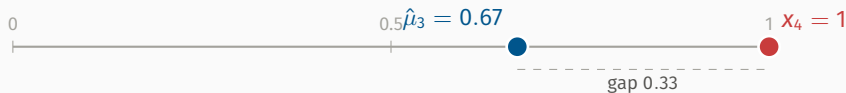


Pattern: new estimate = old estimate + step $\times$ (observation – old estimate). It comes back in TD learning and Q-learning (session 6).

Incremental update

No need to store past rewards: if $\hat{\mu}_n$ is the mean of the first n rewards and x_{n+1} is the next one,

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1} (x_{n+1} - \hat{\mu}_n) .$$

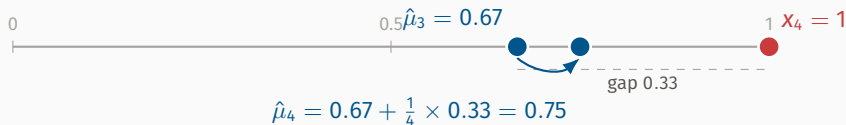


Pattern: new estimate = old estimate + step $\times$ (observation – old estimate). It comes back in TD learning and Q-learning (session 6).

Incremental update

No need to store past rewards: if $\hat{\mu}_n$ is the mean of the first n rewards and x_{n+1} is the next one,

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1} (x_{n+1} - \hat{\mu}_n) .$$

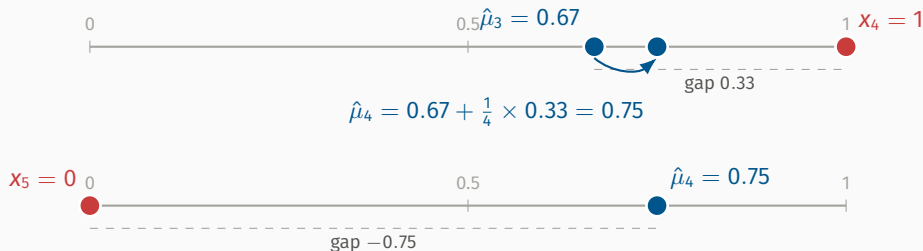


Pattern: new estimate = old estimate + step $\times$ (observation – old estimate). It comes back in TD learning and Q-learning (session 6).

Incremental update

No need to store past rewards: if $\hat{\mu}_n$ is the mean of the first n rewards and x_{n+1} is the next one,

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1} (x_{n+1} - \hat{\mu}_n) .$$

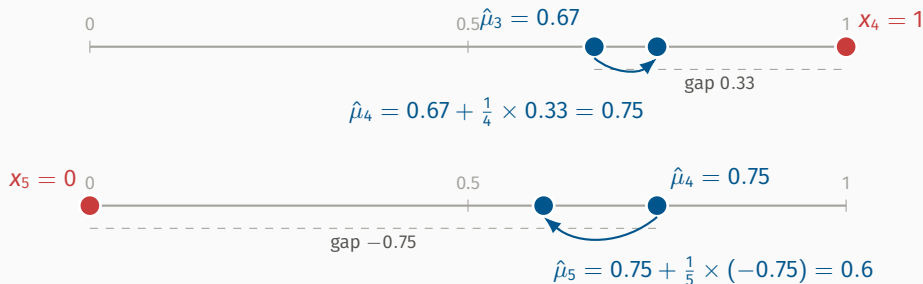


Pattern: new estimate = old estimate + step $\times$ (observation – old estimate). It comes back in TD learning and Q-learning (session 6).

Incremental update

No need to store past rewards: if $\hat{\mu}_n$ is the mean of the first n rewards and x_{n+1} is the next one,

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1} (x_{n+1} - \hat{\mu}_n) .$$



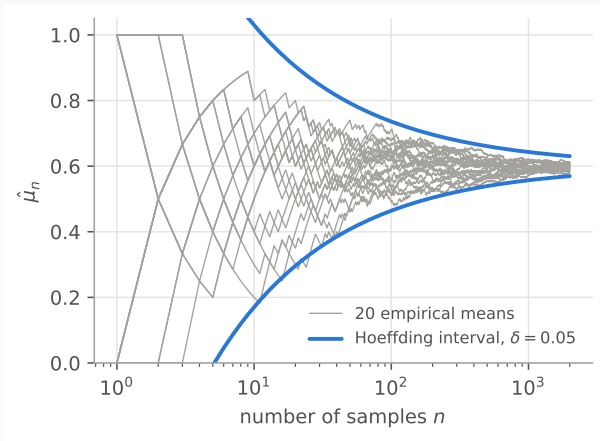
Pattern: new estimate = old estimate + step $\times$ (observation – old estimate). It comes back in TD learning and Q-learning (session 6).

Precision of an empirical mean

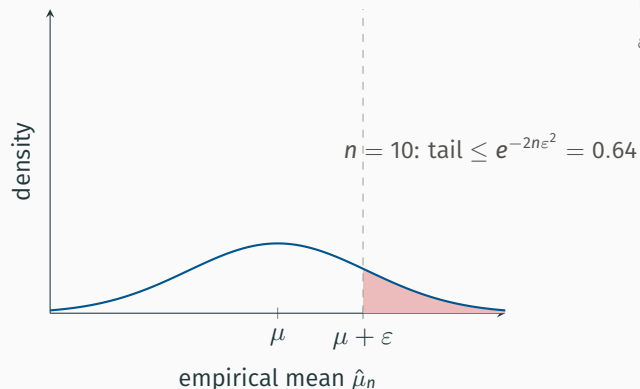
$Y_1, \dots, Y_n$ i.i.d. with mean μ and variance σ^2 :

$$\text{Var}(\hat{\mu}_n) = \frac{\sigma^2}{n} .$$

- The typical error decreases as $1/\sqrt{n}$
- Halving the error takes 4 times more samples
- Every trajectory ends up inside the funnel, but early on anything can happen



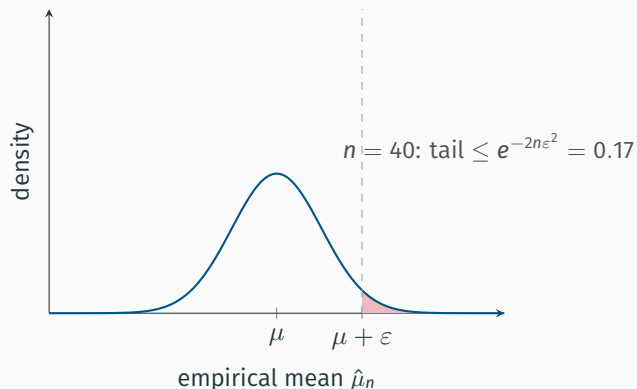
Concentration of the empirical mean



Rewards in $[0, 1]$, fixed deviation $\epsilon = 0.15$:

- the distribution of $\hat{\mu}_n$ concentrates around μ as n grows
- the shaded tail is the probability of overestimating μ by more than ϵ
- Hoeffding: this tail is at most $e^{-2n\epsilon^2}$, **exponentially small** in n , whatever the distribution

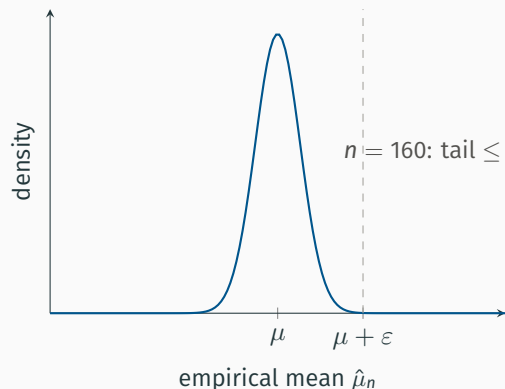
Concentration of the empirical mean



Rewards in $[0, 1]$, fixed deviation $\epsilon = 0.15$:

- the distribution of $\hat{\mu}_n$ concentrates around μ as n grows
- the shaded tail is the probability of overestimating μ by more than ϵ
- Hoeffding: this tail is at most $e^{-2n\epsilon^2}$, **exponentially small** in n , whatever the distribution

Concentration of the empirical mean



Rewards in $[0, 1]$, fixed deviation $\varepsilon = 0.15$:

- the distribution of $\hat{\mu}_n$ concentrates around μ as n grows
- the shaded tail is the probability of overestimating μ by more than ε
- Hoeffding: this tail is at most $e^{-2n\varepsilon^2}$, **exponentially small** in n , whatever the distribution

Hoeffding's inequality

Theorem (Hoeffding, 1963)

Let $Y_1, \dots, Y_n$ be i.i.d. with values in $[\alpha, \beta]$ and mean μ . For any $\varepsilon > 0$,

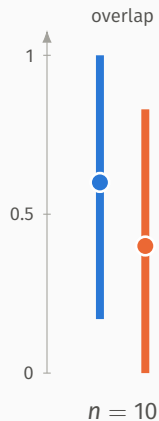
$$\mathbb{P}(\hat{\mu}_n - \mu \geq \varepsilon) \leq \exp\left(-\frac{2n\varepsilon^2}{(\beta - \alpha)^2}\right), \quad \text{and the same holds for } \mathbb{P}(\hat{\mu}_n - \mu \leq -\varepsilon) .$$

Confidence interval. For rewards in $[0, 1]$, with probability at least $1 - \delta$,

$$|\hat{\mu}_n - \mu| \leq \sqrt{\frac{\log(2/\delta)}{2n}} .$$

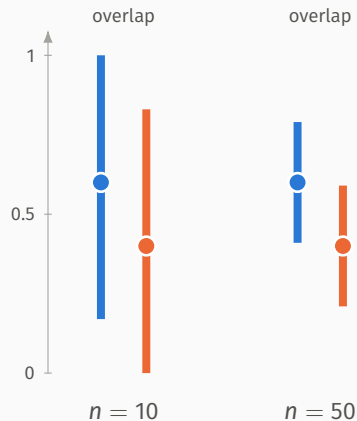
- Example: $n = 100$ and $\delta = 0.05$ give a precision of ± 0.14

Separating two arms



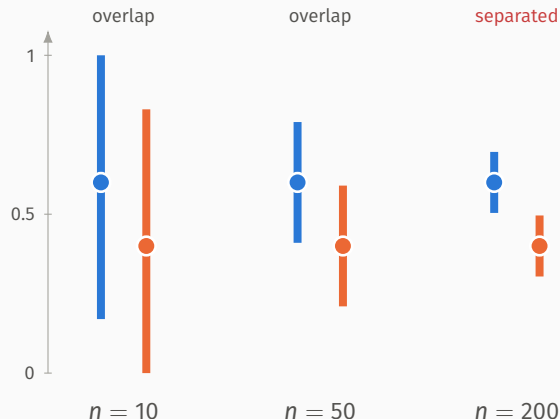
Arm 1: $\mu_1 = 0.6$, arm 2: $\mu_2 = 0.4$,
intervals at $\delta = 0.05$ after n pulls
each.

Separating two arms



Arm 1: $\mu_1 = 0.6$, arm 2: $\mu_2 = 0.4$,
intervals at $\delta = 0.05$ after n pulls
each.

Separating two arms



Arm 1: $\mu_1 = 0.6$, arm 2: $\mu_2 = 0.4$, intervals at $\delta = 0.05$ after n pulls each.

- The intervals separate once $2\sqrt{\log(2/\delta)/(2n)} < \Delta$, that is, $n \gtrsim 185$
- In general, telling apart two arms with gap Δ takes of the order of $\log(1/\delta)/\Delta^2$ pulls
- Halving the gap multiplies the cost by 4

Naive strategies

Uniform exploration

Strategy. A_t uniform on $\{1, \dots, K\}$, ignoring the rewards.



- $\mathbb{E}[N_a(T)] = T/K$ for any arm a , so, by the decomposition,

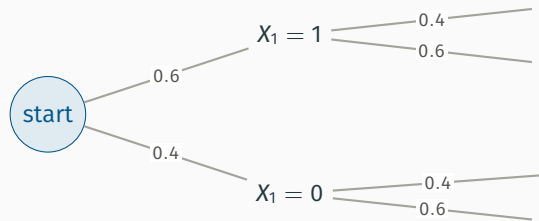
$$R_T = \frac{T}{K} \sum_{a=1}^K \Delta_a ,$$

which is **linear** in T

- We explore forever and never use what we learn

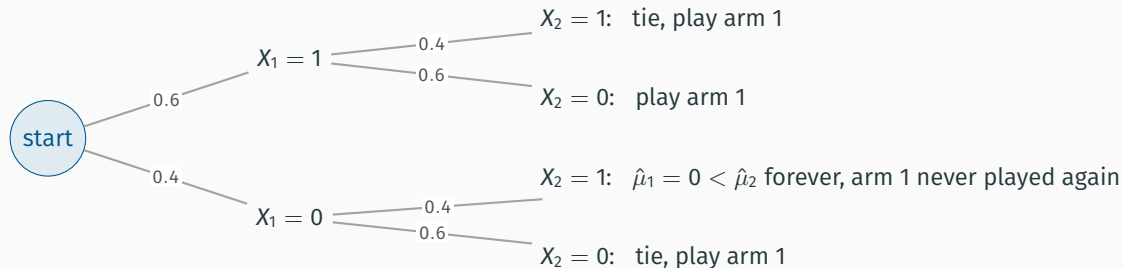
Greedy

Strategy. Pull each arm once, then play $A_t \in \arg \max_a \hat{\mu}_a(t-1)$ (ties go to arm 1). Two Bernoulli arms, $\mu_1 = 0.6$ and $\mu_2 = 0.4$.



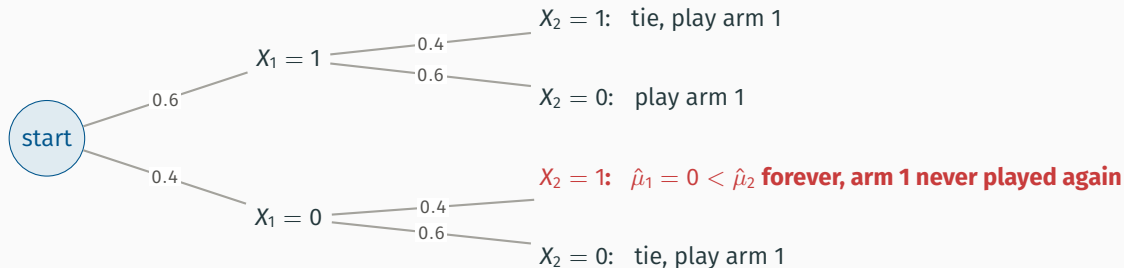
Greedy

Strategy. Pull each arm once, then play $A_t \in \arg \max_a \hat{\mu}_a(t-1)$ (ties go to arm 1). Two Bernoulli arms, $\mu_1 = 0.6$ and $\mu_2 = 0.4$.



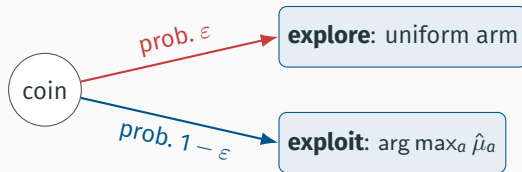
Greedy

Strategy. Pull each arm once, then play $A_t \in \arg \max_a \hat{\mu}_a(t-1)$ (ties go to arm 1). Two Bernoulli arms, $\mu_1 = 0.6$ and $\mu_2 = 0.4$.

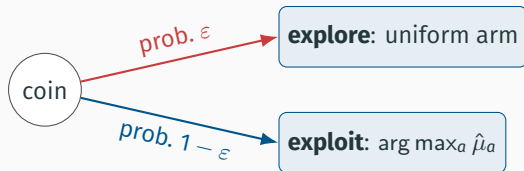


- The locking branch has probability $0.4 \times 0.4 = 0.16$, so $R_T \geq 0.16 \times 0.2 \times (T - 1)$:
linear (tutorial, exercise 2)
- We exploit and never explore

Strategy. At each round, flip a biased coin:

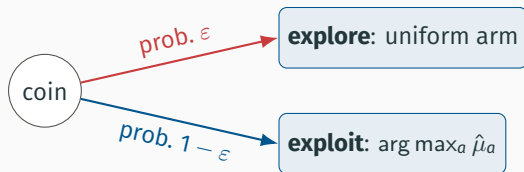


Strategy. At each round, flip a biased coin:



- No more locking: every arm is pulled infinitely often
- But $\mathbb{P}(A_t = a) \geq \epsilon/K$, hence $R_T \geq \frac{\epsilon T}{K} \sum_a \Delta_a$: still **linear** for a fixed ϵ

Strategy. At each round, flip a biased coin:



- No more locking: every arm is pulled infinitely often
- But $\mathbb{P}(A_t = a) \geq \epsilon/K$, hence $R_T \geq \frac{\epsilon T}{K} \sum_a \Delta_a$: still **linear** for a fixed ϵ
- With ϵ_t decreasing as $1/t$, the regret can be logarithmic, but the right tuning depends on the unknown gaps

Explore-then-commit (ETC)

Strategy. Pull each arm m times in turn, then play the empirically best arm until the end. Two arms, arm 1 is the best.



m too small

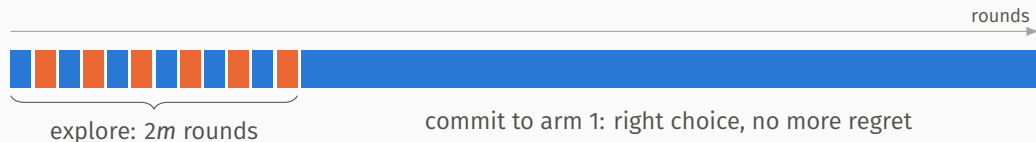
- Imprecise estimates
- We commit to the wrong arm too often

m too large

- Exploration costs $m\Delta$

Explore-then-commit (ETC)

Strategy. Pull each arm m times in turn, then play the empirically best arm until the end. Two arms, arm 1 is the best.



m too small

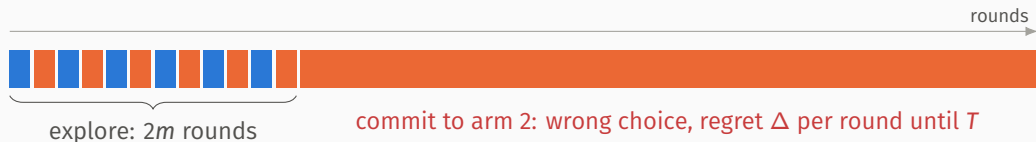
- Imprecise estimates
- We commit to the wrong arm too often

m too large

- Exploration costs $m\Delta$

Explore-then-commit (ETC)

Strategy. Pull each arm m times in turn, then play the empirically best arm until the end. Two arms, arm 1 is the best.



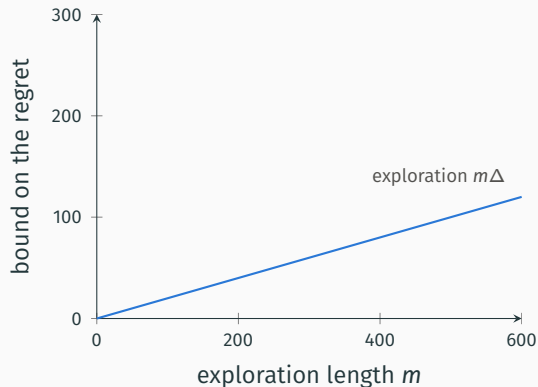
m too small

- Imprecise estimates
- We commit to the wrong arm too often

m too large

- Exploration costs $m\Delta$

Choice of m

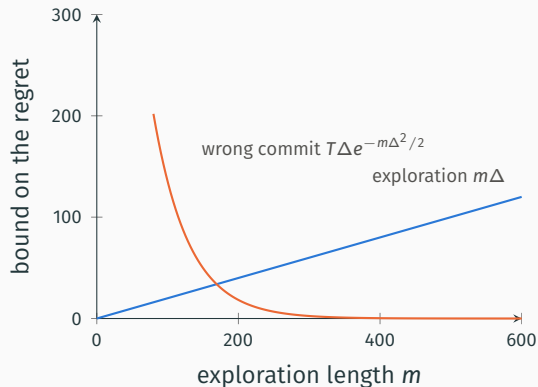


$K = 2$, gap $\Delta = 0.2$, $T = 5000$.

Hoeffding (tutorial, exercise 3)

$$R_T \leq m\Delta + T\Delta \exp(-m\Delta^2/2)$$

Choice of m

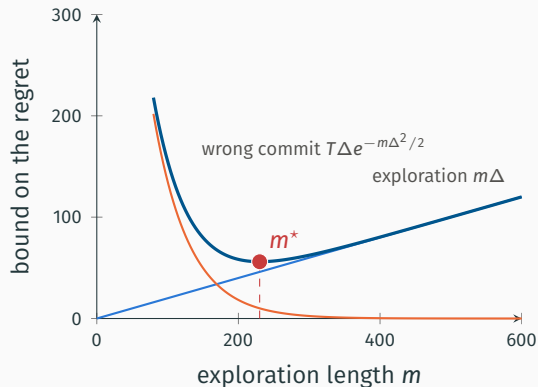


$K = 2$, gap $\Delta = 0.2$, $T = 5000$.

Hoeffding (tutorial, exercise 3)

$$R_T \leq m\Delta + T\Delta \exp(-m\Delta^2/2)$$

Choice of m



$K = 2$, gap $\Delta = 0.2$, $T = 5000$.

Hoeffding (tutorial, exercise 3)

$$R_T \leq m\Delta + T\Delta \exp(-m\Delta^2/2)$$

- Minimum at
 $m^* = \frac{2}{\Delta^2} \log \frac{T\Delta^2}{2} \approx 230$
- Then $R_T \leq \frac{2}{\Delta} (1 + \log \frac{T\Delta^2}{2}) \approx 56$,
against 500 for uniform
exploration

- With the right m , the regret is **logarithmic** in T
- But m^* depends on Δ , which we do not know
- Without knowing Δ , the choice $m \approx T^{2/3}$ guarantees $R_T = O(T^{2/3})$ on every instance (admitted)
- Once committed, ETC never reconsiders its decision
- This is exactly the “how long should the A/B test last?” question of the advertising example

Summary

Strategy	Needs to know	Regret
Uniform exploration	nothing	linear
Greedy	nothing	linear (locking)
ε -greedy, fixed ε	nothing	linear
ETC, $m \approx T^{2/3}$	T	$O(T^{2/3})$
ETC, $m = m^*$	T and Δ	$O(\log(T)/\Delta)$

Session 2

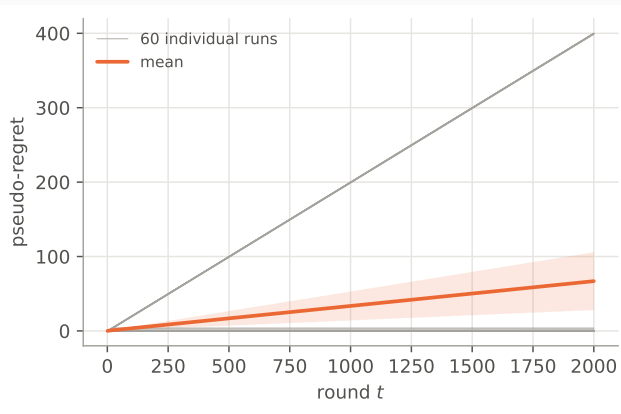
Can we get a $\log T$ regret without knowing Δ ? Yes: UCB and Thompson sampling, which explore according to their uncertainty.

Experimental protocol

Variability across runs

Greedy, $\mu = (0.6, 0.4)$:

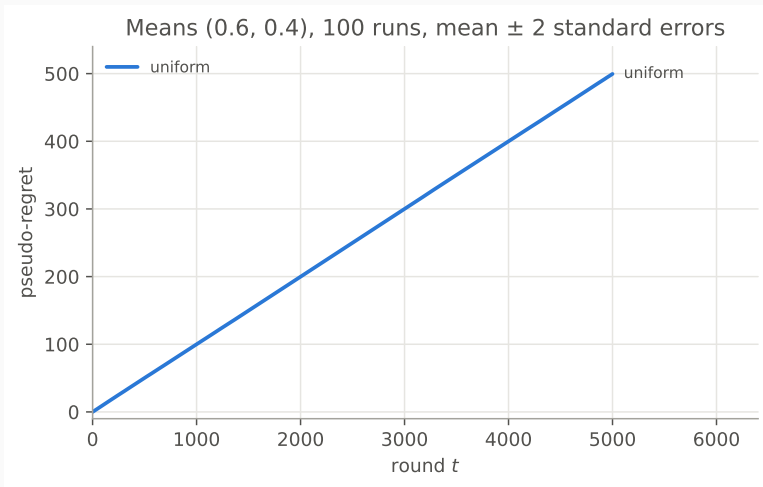
- most runs have an almost flat regret
- a few runs lock onto the wrong arm and have a linear regret
- the average is linear, driven by these few runs



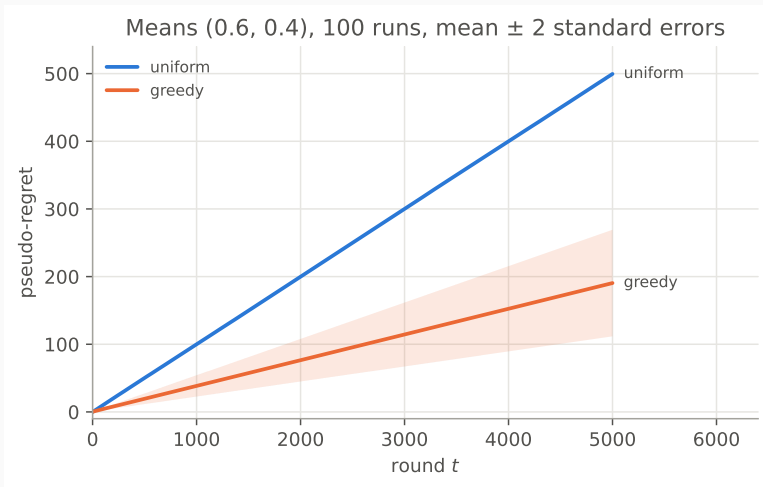
Rules for the experiments

- **Fixed seeds:** every figure can be reproduced exactly
- **Repetitions:** at least 20 independent runs per algorithm
- **Uncertainty bands:** mean ± 2 standard errors ($2\hat{\sigma}/\sqrt{n_{\text{runs}}}$), or quantiles
- **Fair comparisons:** same instance, same horizon, same environment seeds
- **Stated instance:** arm means and horizon written on the figure

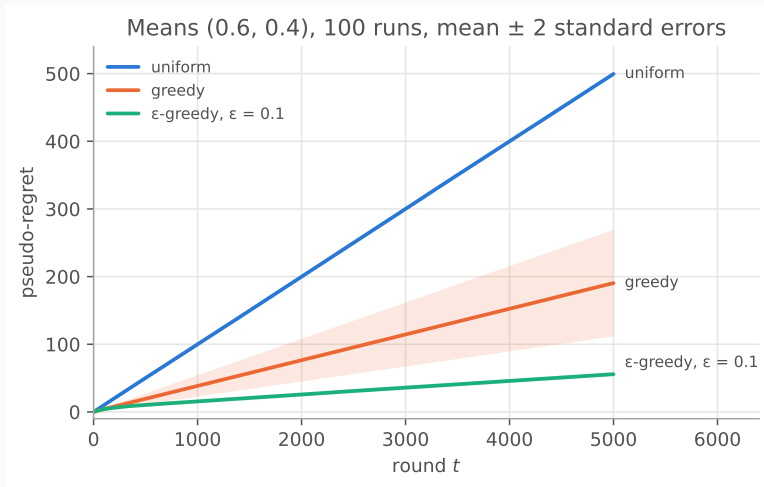
Comparison of the naive strategies



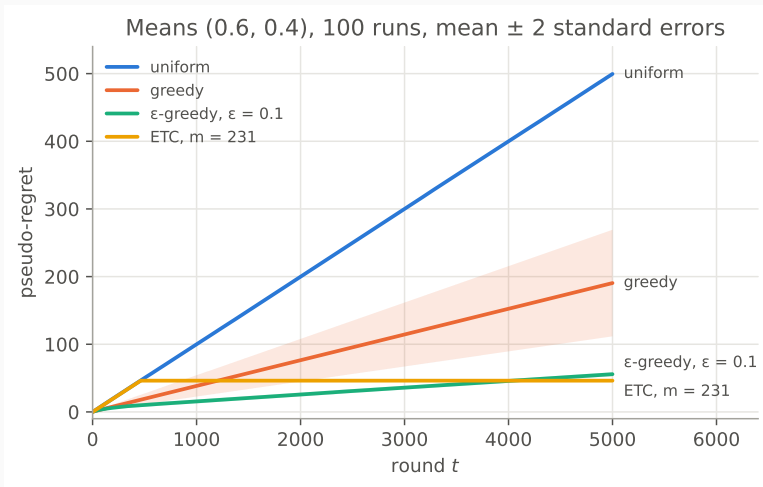
Comparison of the naive strategies



Comparison of the naive strategies



Comparison of the naive strategies



Tutorial 1

- Regret decomposition
- Locking of greedy
- Tuning ETC with Hoeffding

Lab 0 (if Python or numpy is new to you)

- Installation, numpy, matplotlib, classes

Lab 1

- Getting started with seqlearn
- Greedy, ϵ -greedy, ETC
- Checking the tutorial results numerically

Reading

- Lattimore and Szepesvári, *Bandit Algorithms*, chapters 4 and 6
- Slivkins, *Introduction to Multi-Armed Bandits*, chapter 1